

# Visitor Counting Console

## Features and technical specification

30 September 2026

# What it is

Visitor counting on the CCTV a venue already has.

- ▶ A trigger line drawn across each entrance
- ▶ A person is counted when their **head point** crosses it
- ▶ Zero-shot classification gives **male / female** and **adult / child**
- ▶ No turnstile, no new hardware, no app

One machine with a GPU runs everything. No database, no cloud, no per-site training.

# How it works

RTSP / HTTP / file

- > threaded decode, one thread per source
- > YOLO11 detection, one batched pass for all cameras
- > ByteTrack, one tracker per camera
- > head point crosses the line -> entry (+1) / exit (-1)
- > SigLIP 2 gender and adult/child on selected crops
- > counters, alerts, CSV logs
- > MJPEG overlays + JSON state over WebSocket

## Two figures, deliberately different

- ▶ **Visitors today** counts entries only and never falls
- ▶ **Current occupancy** is net and falls again when people leave

Someone who walks in and out has still visited.

Conflating the two is the most common way a footfall dashboard misleads.

## Counting: the parts that stop a wrong number looking right

- ▶ **8 % tolerance** past each end of the line — and it is *drawn*, because an invisible tolerance is indistinguishable from a bug
- ▶ **4 % edge warning** — a line too near the frame edge fires before the tracker has established the person
- ▶ **Hysteresis + multi-frame confirmation** absorb detector jitter
- ▶ **One event per side transition per track**; a new track at the same spot is treated as an ID switch and ignored

## Demographics: two attributes, two strategies

- ▶ **Gender** — last three looks plus one at the moment of crossing, counted double. The read swings early in a track and settles at the line.
- ▶ **Age** — averages every observation. “Child” is noisy per frame and smooths out.

Measured on 56 hand-labelled crossings:

| attribute     | averaging | split strategy |
|---------------|-----------|----------------|
| gender        | 90.9 %    | <b>94.5 %</b>  |
| adult / child | 96.4 %    | <b>100 %</b>   |

## Counting areas have a role

Restricting what is **tracked** and what is **classified** are different needs.

| role                 | effect                                                  |
|----------------------|---------------------------------------------------------|
| count only inside    | only people inside are tracked at all                   |
| ignore inside        | people inside are skipped: a window, a mirror, a poster |
| classify only inside | everyone counted, only those inside sexed and aged      |

Only the first constrains where the line may go — and the console says so when it is violated.

# An area reports its own population

Who is standing in it **right now**, split male/female and adult/child.

- ▶ Owes nothing to the trigger line
- ▶ Reads correctly from the first frame rather than starting at zero
- ▶ Undecided people are reported as unknown, so the parts always sum to the total

*Classify only inside* is the accuracy lever on a wide scene: spend the crop budget where people are large and clear.

# Cameras

- ▶ Up to **six**, added and removed while running; the grid re-lays itself out
- ▶ RTSP, HTTP or a video file per camera
- ▶ Passwords with @ : \$ % & or spaces are encoded automatically
- ▶ **Paired feeds**: one key swaps a panel between its live camera and a saved clip
- ▶ An unreachable camera reports OFFLINE and **hides its figures** rather than showing frozen ones

## Capacity, alerts, evidence

- ▶ Site capacity with a warning threshold; the banner **names the entrance** that tipped the count
- ▶ Per-area capacity, alarming independently
- ▶ One button records 10 s before and 20 s after, with a note
- ▶ Over capacity records a clip automatically
- ▶ Every crossing is logged with the probabilities behind the decision

# Interfaces

- ▶ **24 HTTP and WebSocket endpoints** — state, history, cameras, lines, areas, capacity, recording
- ▶ **MJPEG** overlay stream per camera, encoded once and shared between viewers
- ▶ **JSON state** pushed over WebSocket at about 4 Hz
- ▶ `output/events.csv` — one row per crossing
- ▶ `output/history.csv` — hourly, closing, corrections, events

# Configuration stays on the machine

Three JSON files, written by the application, excluded from source control, seeded from committed templates:

- ▶ `sources.json` — the live and video source sets
- ▶ `lines.json` — trigger lines and counting areas, in normalised coordinates
- ▶ `site.json` — hours, capacity, thresholds

A source update can never collide with a machine's own calibration, and camera credentials never reach the repository.

# Platforms and performance

| platform              | acceleration |
|-----------------------|--------------|
| Ubuntu 22.04 / 24.04  | NVIDIA CUDA  |
| Windows 10 / 11       | NVIDIA CUDA  |
| macOS (Apple Silicon) | Apple MPS    |
| Ubuntu + Docker       | NVIDIA CUDA  |

RTX 3060 Ti, three 1080p streams: detection and tracking ~40 ms per iteration, classification ~10-25 ms, **~11-14 iterations per second**. Throughput divides across cameras.

# Known limits

- ▶ **Counting is the robust part.** It needs the camera to see heads cleanly across the line — overhead or high-angle works best.
- ▶ **Zero-shot classification is a baseline**, not a purpose-trained attribute model.
- ▶ **Distant people are counted but not classified**, and are reported as unknown rather than guessed.
- ▶ **The detector is AGPL-3.0.** Offering this as a network service may oblige disclosure or a commercial licence. One file imports it.

# Summary

- ▶ Runs on existing CCTV, one machine, no cloud
- ▶ Two figures that mean different things, and never conflated
- ▶ Accuracy work is measured, not asserted
- ▶ Unknowns are reported, not guessed
- ▶ Every number auditable back to the frames that produced it